

day02笔记

使用命令必须养成的习惯

- 1 1、tab键自动补全
- 2 2、Ctrl + l : 清理屏幕
- 3 3、Ctrl + c : 终止当前命令的执行

常用远程连接软件

- 1 # 终端仿真程序, 其实就是Windows下登录UNIX或Linux服务器主机的软件, 支持ssh、telnet
- 2 1、Xshell
- 3 2、Secure CRT
- 4
- 5 # xshell实现文件互传
- 6 1、xshell图形界面: 新建文件传输
- 7 2、安装: lrzsz,是一款可在linux里可代替ftp上传和下载的程序

最最最常用命令

- 1 1、ps -aux | grep servername
- 2 2、sudo kill PID
- 3 3、chmod 644 filename
- 4 4、chown user:group filename
- 5 5、ssh user@IP
- 6 6、scp filename user@IP:directory
- 7 7、df -h
- 8 8、find directory -name filename

周期性计划任务

- 1 # 1、进入周期性计划任务
- 2 crontab -e (首次进入按2 - 找vim)
- 3
- 4 # 设置周期性计划任务
- 5 * * * * * : 五个*号代表 分 时 日 月 周

```

6 分：0-59
7 时：0-23
8 日：1-31
9 月：1-12
10 周：0-6
11
12 # 开始设置：
13 1、'*' 代表所有可能值
14 2、',' 指定多个时间点
15 3、 '/' 指定时间间隔频率
16 4、 '-' 指定一个时间段
17
18 # 示例
19 1、每月的1日和5日两天：* * 1,5 * *
20 2、每10分钟：*/10 * * * *
21 3、0点-6点每小时执行：0 0-6/1 * * *
22 4、每分钟执行：* * * * *
23
24 # 练习
25 1、每小时的第3分钟和第15分钟执行
26 3,15 * * * *
27 2、每周六、周日的0点执行一个 01.py 文件
28 0 0 * * 6,0
29 6、每天18:00到23:00之间每小时执行 01.py 文件
30 0 18-23/1 * * *

```

文本处理工具 - awk

语法格式

```
1 | awk 选项 '动作' 文件列表
```

常用方式

```
1 | Linux命令 | awk 选项 '动作'
```

使用方法

```

1 # 示例
2 awk '{print "abc"}' ip.txt
3 # 思考：这个会输出什么？
4 df -h | awk '{print $1}'
5
6 # -F：指定分隔符
7 awk -F ":" '{print $2}' # 显示：分隔后的第2列
8
9 # 示例1：输出当前Linux操作系统的所有用户
10 # 练习：
11 输出本机的IP地址
12

```

练习

```
1 # nginx的访问日志目录 : /var/log/nginx/access.log
2 问题1: 把访问过自己的IP地址输出
3     # awk '{print $1}' access.log
4 问题2: 统计有多少个IP访问过我
5     # awk '{print $1}' access.log | sort | uniq | wc -l
6 问题3: 统计每个IP地址的访问次数,输出前10个访问量最大的用户IP
7     # awk '{print $1}' access.log | sort | uniq -c | sort -rn -k 1 | head -10
```

文本处理工具 - sed

```
1 # 1、定义
2 非交互式文本编辑器,逐行处理
3
4 # 2、语法格式
5 sed [选项] '条件指令' 文件名
6 指令: 增删改查
7
8 # 查 - p
9 sed -n 'p' file.txt      # 打印所有内容
10 sed -n '1p' file.txt    # 打印第1行内容
11 sed -n '1,3p' file.txt  # 打印1-3行内容
12 sed -n '1p;3p' file.txt # 打印1和3行内容
13
14 # 删 - d
15 sed -i '1d' file.txt    # 删除第1行内容
16 sed -i '$d' file.txt    # 删除最后1行内容
17 sed -i '1,3d' file.txt  # 删除第1-3行内容
18 sed -i '1d;3d' file.txt # 删除第1和第3行内容
19
20 # 改 - c
21 sed -i 'c内容' file.txt # 修改所有行内容
22 sed -i '1c内容' file.txt # 修改第1行内容
23 sed -i -e '1c李白' -e '3c杜甫' file.txt
24 # 修改第1行为李白,修改第3行为杜甫
25
26 # 增加 - a i
27 a: 在当前处理行的下一行插入
28 i: 在当前处理行的上一行插入
29 sed -i 'a李白' file.txt # 每一行的下一行都李白
30 sed -i '1a李白' file.txt
31 sed -i '3i杜甫' file.txt
32
33 # 增-a 删-d 改-c 查-p
34
35 # 替换 - s
36 sed -i '条件/原内容/新内容/选项' file.txt
37 选项: g 表示全局替换 i 忽略字母大小写
38 # 只替换每行中第一个李白
39 sed -i '1,3s/李白/李清照/' file.txt
40 # 替换每行中所有李白,加 g 选项
41 sed -i '1,3s/李白/李清照/g' file.txt
```

grep命令之正则表达式

```
1 # 正则表达式元字符集 - 使用grep命令
2 ^      : 以 ... 开头
3 $      : 以 ... 结尾
4 .      : 任何1个字符
5 *      : 0次或多次
6
7 # 正则表达式扩展字符集 - 使用 egrep 命令
8 +      : 1次或多次
9 {n}    : 出现n次
10 ()     : 分组
11
12 [a-z]  : 所有小写字母
13 [A-Z]  : 所有大写字母
14 [a-Z]  : 所有字母
15 [0-9]  : 所有数字
16 [a-Z0-9] : 所有的字母和数字
```

应用场景

```
1 # Mac地址正则匹配
2 ([0-9a-fA-F]{2}:){5}[0-9a-fA-F]{2}
3 ifconfig | egrep "([0-9a-fA-F]{2}:){5}[0-9a-fA-F]{2}" | awk '{print $2}'
4
5 # 批量处理headers和formdata的正则表达式?
6 (.*): (.*)
```

shell编程

Shell格式

```
1 1、扩展名: xxx.sh
2 2、正文第一行必须指定解释器: #!/bin/bash
```

shell执行方式

```
1 # 方式一：加权限， ./xxx.sh 执行
2 1、 chmod +x xxx.sh
3 2、 ./xxx.sh
4
5 # 方式二：手动指定解释器
6 bash xxx.sh
```

变量

▪ 自定义变量

```
1 # 1. 定义变量
2 变量名=值    ----> 注意：=两侧绝对不能有空格
3 eg1: name="take me to your heart"
4
5 # 2. 调用变量的格式
6 echo $变量名
7
8 # 3. 小细节：单引号和双引号的区别
9 单引号：无法获取变量的值
10 双引号：可以获取变量的值
```

▪ 环境变量+位置变量+预设变量

```
1 # 环境变量
2 echo $USER    -- 当前用户
3 echo $UID     -- 当前用户的UID号
4 echo $PWD     -- 当前路径
5 echo $PATH    -- 命令搜索路径
6
7 # 位置变量
8 $1 $2 $3 ... .. shell的位置变量
9
10 # 预定义变量
11 $# $* $?
12
13 # $? : 返回上一条命令执行的状态(0代表正确,非0代表失败)
```

示例

```
1 输出$1+$2,例如输出结果：3+5
2 #!/bin/bash
3 echo $1+$2
```

▪ 变量赋值 - 接收用户从终端输入的值

```
1 # 语法规则
2 read -p 提示信息 变量名
3
4 # 示例
```

```

5 #!/bin/bash
6 read -p 请输入姓名: name
7 echo "您输入的姓名是:$name"
8
9 # 指定超时时间
10 read -p 提示信息 变量名
11 read -t n -p 提示信息 变量名
12
13 # 示例
14 #!/bin/bash
15 read -t 3 -p 请输入用户名: username

```

练习

- 1 1、输入学生姓名: 赵敏
- 2 2、输入学生成绩: 88
- 3 3、输出: 赵敏的成绩为88分

shell - 算术运算符

```

1 # 运算符
2 1、+ - * / %
3 2、++ : 自加1运算,类似于python中 i++ 等同于 i+=1
4 3、-- : 同++
5
6 # 运算命令
7 1、let 运算表达式
8   i=1
9   let i++
10  echo $i
11 2、expr 运算表达式
12   i=1
13   sum=`expr $i + 5` # +两侧要有空格
14   echo $sum
15 3、$[]
16   echo ${1+1}
17   echo ${1-1}
18   echo ${a+a} # 调用变量不用多次添加$符号
19   echo ${1*1} # 乘法无需转义

```

练习

- 1 使用 位置变量+以上方法一、二中任何一种,实现2个数字的相加
- 2 #!/bin/bash
- 3 echo \${1+2}
- 4 echo `expr \$1 + \$2`

shell - 比较运算符

```
1 # 语法格式
2 [ 判断语句 ] # 注意括号必须有空格
3
4 # 1、字符比较
5 [ A == A ] #相等(等号两边需要有空格)
6 [ A != B ] #不相等
7 [ -z $变量 ] #判断是否为空
8
9 # 2、数字比较
10 -eq 等于(equal)
11 -ne 不等于(not equal)
12 -gt 大于(greater than)
13 -ge 大于等于(great or equal)
14 -lt 小于(less than)
15 -le 小于等于(less or equal)
16
17 # 3、文件|目录比较
18 [ -e 文件或目录 ] #是否存在exist
19 [ -f 文件 ] #存在且为文件file
20 [ -d 目录 ] #存在且为目录directory
21 [ -r 文件或目录 ] #判断是否可读read
22 [ -w 文件或目录 ] #判断是否可写write
23 [ -x 文件或目录 ] #判断是否可执行
```

shell - if分支结构

```
1 # 1、单分支语法格式
2 if 判断 ;then
3     命令
4     命令
5 fi
6 # 2、双分支语法格式
7 if 判断 ;then
8     命令1
9 else
10    命令2
11 fi
12 # 3、多分支语法格式
13 if 判断;then
14     命令1
15 elif 判断 ;then
16     命令2
17 else
18     命令3
19 fi
20 # 示例
21 #!/bin/bash
22 if [ $USER == tarena ];then
23     echo "Yes,You are Tarena."
```

```
24 else
25     echo "You are other man."
26 fi
```

练习:使用shell编写猜数字游戏,无须循环

```
1  #!/bin/bash
2  num=$RANDOM
3  read -p "我有一个随机数,你猜:" guess
4  if [ $guess -eq $num ];then
5      echo "恭喜,猜对了."
6      exit
7  elif [ $guess -gt $num ];then
8      echo "你猜大了"
9  else
10     echo "你猜小了"
11 fi
```

*shell - for*循环

```
1  # 语法格式
2  for 变量 in 值序列
3  do
4      命令
5  done
6  # 示例
7  for i in 1 2 3 4 5
8  do
9      echo "hello world"
10 done
```

练习:判断指定网段的IP地址哪些可以用,哪些不能用?

```
1  #!/bin/bash
2
3  for i in {1..254}
4  do
5      ping -c 2 172.40.91.$i &>/dev/null
6      if [ $? -eq 0 ];then
7          echo "172.40.91.$i is up."
8      else
9          echo "172.40.91.$i is down"
10     fi
11 done
```

*shell - while*循环


```

1 # 语法格式
2 while 条件判断
3 do
4     命令
5 done
6
7 # 示例
8 #!/bin/bash
9 i=1
10 while [ $i -lt 5 ]
11 do
12     echo baby
13     let i++
14 done

```

shell - case分支结构

```

1 # 1、特点
2 根据变量值的不同,执行不同的操作
3
4 # 2、语法格式
5 case $变量名 in
6 模式1)
7     代码块
8     ;;
9 模式2)
10    代码块
11    ;;
12 *)
13    代码块
14    ;;
15 esac

```

示例 - 输入一个字符,判断是数字、字母还是其他字符

```

1 #!/bin/bash
2
3 while :
4 do
5     echo "++++++++++++++++++++++++++++++++++++"
6     echo "    Welcome(q to quit)    "
7     echo "++++++++++++++++++++++++++++++++++++"
8
9     read -p "请输入一个字符:" char
10    if [ ${#char} != 1 ];then
11        echo "${char}不是一个字符"
12    elif [ $char == 'q' ];then
13        echo "程序退出"
14        exit
15    fi
16
17    case $char in

```

```

18 [a-z]|[A-Z])
19     echo "字母"
20     ;;
21 [0-9])
22     echo "数字"
23     ;;
24 *)
25     echo "其他字符"
26     ;;
27 esac
28 done

```

练习:编写1个nginx的启动脚本, 包含: start stop restart

```

1 #!/bin/bash
2
3 read -p "操作(start|stop|restart):" op
4 case $op in
5     "start")
6         sudo /etc/init.d/nginx restart
7         ;;
8     "stop")
9         sudo /etc/init.d/nginx stop
10        ;;
11    "restart")
12        sudo /etc/init.d/nginx restart
13        ;;
14    *)
15        echo "Please choice in start|stop|restart"
16        ;;
17    esac

```

Ubuntu设置sudo免密码

```

1 # 通过更改 /etc/sudoers 实现
2 1、备份: sudo cp /etc/sudoers .
3 2、修改: sudo vi /etc/sudoers
4 添加: tarena ALL=(ALL) NOPASSWD : ALL

```

知识点总结

```

1 # 1、获取字符串长度
2 ${#变量名}
3
4 # 2、字符串索引及切片
5 ${string:index:number}
6 key='ABCDE'
7 ${key:0:1} # A 获取下标索引为0的元素
8 ${key:1:2} # BC
9
10 # 3、vim批量缩进
11 1、进入命令行模式 : shift + :
12 2、1,3> + Enter : 1-3行缩进
13 3、1,3< + Enter : 1-3行往回缩进

```

shell实战

1、每2秒中检测一次MySQL数据库的连接数量

```
1 # mysqladmin命令
2 mysql服务器管理任务的工具，它可以检查mysql服务器的配置和当前工作状态
```

代码实现

```
1 #!/bin/bash
2 #每2秒检测一次MySQL并发连接数
3
4 user="root"
5 passwd="123456"
6
7 while :
8 do
9     sleep 2
10    count=`mysqladmin -u"$user" -p"$passwd" status | awk '{print $4}'`
11    echo "`date %F` 并发连接数为:$count"
12 done
```

2、根据md5校验码，检测文件是否被修改

```
1 # 1、生成md5的文件校验码
2 md5sum nginx.conf
```

代码实现

```
1 #!/bin/bash
2 #本示例脚本检测的是/etc 目录下所有的conf结尾的文件
3 #本脚本在目标数据没有被修改时执行一次，当怀疑数据被人篡改，再执行一次
4 #将两次执行的结果做对比，MD5码发生改变的文件，就是被人篡改的文件
5 for i in $(ls /etc/*.conf)
6 do
7     md5sum "$i" >> /home/tarena/md5log.txt
8 done
```

3、备份MySQL数据库

```
1 # 备份MySQL数据库中的mysql库
2 #!/bin/bash
3
4 user="root"
5 passwd="123456"
6 dbname="mysql"
7 date=$(date +%Y%m%d)
8
9 #测试备份目录是否存在，不存在则自动创建该目录
10 if [ ! -d /home/tarena/mysqlbackup ];then
11     mkdir /home/tarena/mysqlbackup
12 fi
```

```

13
14 #使用mysqldump命令备份数据库
15 mysqldump -u"$user" -p"$passwd" "$dbname" > /home/tarena/mysqlbackup/"$dbname"-${date}.sql
16

```

4、随机生成8为密码

```

1 #!/bin/bash
2 #设置变量key，存储密码的所有可能性（密码库），如果还需要其他字符请自行添加其他密码字符
3 #使用${#key}统计密码库的长度
4
5 key="0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
6 num=${#key}
7 #设置初始密码为空
8 pass=''
9 #循环8次，生成 8为随机密码
10 #每次都是随机数对密码库的长度取余，确保提取的密码字符不超过密码库的长度
11 #每次循环提取一位随机密码，并将该随机密码追加到pass变量的最后
12 for i in {1..8}
13 do
14     index=$((RANDOM%num))
15     pass=$pass${key:index:1}
16 done
17 echo $pass

```

shell - 函数

```

1 # 1、语法格式
2 函数名(){
3     代码块
4 }
5 函数名 # 函数调用,不能加()
6
7 # 2、示例：打印10个*
8 star(){
9     echo "*****"
10 }
11 star # 第1次调用
12 star # 第2次调用
13
14 # 3、练习：写1个计算器程序,计算 加 减 即可 -- 函数+case
15 #!/bin/bash
16
17 sumx(){
18     echo $[n1+n2]
19 }
20
21 subx(){
22     echo $[n1-n2]
23 }
24
25

```

```

26
27 read -p "输入第一个数字: " n1
28 read -p "输入第二个数字: " n2
29 read -p "选择操作(+|-):" op
30
31
32 case $op in
33     "+")
34         sumx
35         ;;
36     "-")
37         subx
38         ;;
39     *)
40         echo "Invalid"
41         ;;
42 esac

```

练习

```

1  在用户主目录下创建一个目录,如果存在则提示,否则提示创建成功
2  #!/bin/bash
3  is_directory(){
4      read -p "请输入要创建的目录名称:" dir
5      if [ -d /home/tarena/$dir ];then
6          echo "该目录已存在"
7      else
8          mkdir /home/tarena/$dir
9          echo "目录 /home/tarena/$dir 创建成功"
10     fi
11 }
12 is_directory

```

字符串处理

用法

```
1  ${变量名 替换符号 匹配条件}
```

从左向右删除

```

1  # 1、语法
2  ${变量名##匹配条件}
3
4  # 2、示例
5  directory="/home/tarena/mysql" # 注意{}中不需要加空格
6  echo ${directory##*/}    --> mysql
7  echo ${directory#*/}     --> home/tarena/mysql

```

从右向左删除

```
1 # 1、语法
2 ${变量名%%匹配条件}
3
4 # 2、示例
5 directory="/home/tarena/mysql"
6 echo ${directory%/mysql} --> /home/tarena
7 echo ${directory%/*} --> /home/tarena
8 echo ${directory%/*} --> ""
```

案例

```
1 输出系统中的前10个用户
2
3 #!/bin/bash
4 for filename in `head -10 /etc/passwd`
5 do
6     echo ${filename%:*}
7 done
8
9 # 方法2
10 head -10 /etc/passwd | awk -F ':' '{print $1}'
11
12 # 方法3
13 使用sed命令替换
14 sed 's/:.*//' /etc/passwd | head -10
```

练习

```
1 批量修改文件名：把当前目录下的.txt文件全部改为.doc文件
2
3 #!/bin/bash
4 for filename in `ls *.txt`
5 do
6     name=${filename%.txt}
7     mv $filename $name.do
8 done
```

Python重要知识点回顾

Python中的那些锁

■ GIL锁

```
1 # 1. GIL是什么? - CPython
2 全局解释器锁,限制多线程同时执行,保证同一时间内只有一个线程在执行
3
4 # 2. 作用
5 同一进程中线程是数据共享,当各个线程访问数据资源时会出现竞争状态,即数据可能会同时被多个线程占用,造成数据混乱,这就是线程的不安全。而解决多线程之间数据完整性和状态同步最简单的方式就是加锁。GIL能限制多线程同时执行,保证同一时间内只有一个线程在执行
6
7 # 3. 影响
8 影响多线程效率
9
10 # 4. 如何避免?
11 1、用进程 代替 多线程
12 2、更换解释器
```

▪ 互斥锁

```
1 # 1. 问题原因
2 多个线程共享数据时,如果数据不进行保护,则可能出现数据不一致现象
3 # 2. 解决方案
4 使用一把锁把代码保护起来,以牺牲性能换取代码的安全性
5 # 3. 示例
```

▪ 死锁

```
1 # 1.定义
2 多个进程/线程在执行过程中,因争夺资源而造成的一种互相等待的现象,若无外力作用,它们都无法进行下去,此时称系统处于死锁状态
3 # 2.示例
4
5 # 3.解决
```